Overview

The goal of this document is to provide comprehensive reference documentation for programmers writing tests, extension authors, and engine authors as well as build tool and IDE vendors.

What is JUnit?

JUnit is composed of several different modules from three different sub-projects.

JUnit 6.1.0 = *JUnit Platform* + *JUnit Jupiter* + *JUnit Vintage*

The **JUnit Platform** serves as a foundation for launching testing frameworks on the JVM. It also defines the `TestEngine` API for developing a testing framework that runs on the platform. Furthermore, the platform provides a Console Launcher to launch the platform from the command line and the JUnit Platform Suite Engine for running a custom test suite using one or more test engines on the platform. First-class support for the JUnit Platform also exists in popular IDEs (see IntelliJ IDEA, Eclipse, NetBeans, and Visual Studio Code) and build tools (see Gradle, Maven, Ant, Bazel, and sbt).

JUnit Jupiter is the combination of the programming model and extension model for writing JUnit tests and extensions. The Jupiter sub-project provides a `TestEngine` for running Jupiter based tests on the platform.

JUnit Vintage provides a `TestEngine` for running JUnit 3 and JUnit 4 based tests on the platform. It requires JUnit 4.12 or later to be present on the class path or module path. Note, however, that the JUnit Vintage engine is deprecated and should only be used temporarily while migrating tests to JUnit Jupiter or another testing framework with native JUnit Platform support.

Supported Java Versions

JUnit requires Java 17 (or higher) at runtime. However, you can still test code that has been compiled with previous versions of the JDK.

Getting Help

Ask JUnit-related questions on Stack Overflow or use the Q&A category on GitHub Discussions.

Getting Started

Downloading JUnit Artifacts

To find out what artifacts are available for download and inclusion in your project, refer to Dependency Metadata. To set up dependency management for your build, refer to Build Support and the Example Projects.

JUnit Features

To find out what features are available in JUnit 6.1.0 and how to use them, read the corresponding sections of this User Guide, organized by topic.

- Writing Tests in JUnit Jupiter
- Migrating from JUnit 4 to JUnit Jupiter
- Running Tests
- Extension Model for JUnit Jupiter
- Advanced Topics
  - JUnit Platform Launcher API
  - JUnit Platform Test Kit

Example Projects

To see complete, working examples of projects that you can copy and experiment with, the `junit-examples` repository is a good place to start. The `junit-examples` repository hosts a collection of example projects based on JUnit Jupiter, JUnit Vintage, and other testing frameworks. You'll find appropriate build scripts (e.g., `build.gradle`, `pom.xml`, etc.) in the example projects. The links below highlight some of the combinations you can choose from.

- For Gradle and Java, check out the `junit-jupiter-starter-gradle` project.
- For Gradle and Kotlin, check out the `junit-jupiter-starter-gradle-kotlin` project.
- For Gradle and Groovy, check out the `junit-jupiter-starter-gradle-groovy` project.
- For Maven, check out the `junit-jupiter-starter-maven` project.
- For Ant, check out the `junit-jupiter-starter-ant` project.
- For Bazel, check out the `junit-jupiter-starter-bazel` project.
- For sbt, check out the `junit-jupiter-starter-sbt` project.